Module 5 — Assembly Language Machine Problems

NASM 32-bit | Procedures | Conditional Selection | User Input | ASCII Art

Two interactive console applications • Intermediate difficulty • Recommended time: 2–3 hours per problem

General Programming Requirements

Develop both applications using NASM x86 32-bit assembly on Linux with int 0x80. Programs must compile and run without errors.

Requirement	Minimum
Procedures using CALL and RET	5
User input operations	3
Conditional selections	5
Menu options	4
ASCII art designs	3
Menu repetition or loop	1
Input validation and exit option	Required

Machine Problem 1 — Zodiac Horoscope and Fortune Teller

Problem Statement

Create an interactive Zodiac Horoscope and Fortune Teller System. The user enters a name, birth month and birth day. The program identifies the zodiac sign and displays a fictional horoscope according to a selected category. Separate major tasks into procedures and use conditional branches for date and menu decisions.

1. Welcome Screen

```
=======================================
       THE MYSTIC ZODIAC ORACLE
=======================================
             *       *
          *     /\      *
               /  \
         *    /____\    *
              |  |
            *   |__|   *
       DISCOVER YOUR DESTINY!
=======================================
```

2. User Information

```
Enter your name: Maria
Enter birth month (1-12): 8
Enter birth day (1-31): 15

Welcome, Maria!
```

Validate month and day against actual month lengths. February 29 may be rejected because no birth year is collected.

3. Zodiac Identification

Sign	Date range
Capricorn	Dec 22–Jan 19
Aquarius	Jan 20–Feb 18
Pisces	Feb 19–Mar 20
Aries	Mar 21–Apr 19
Taurus	Apr 20–May 20
Gemini	May 21–Jun 20
Cancer	Jun 21–Jul 22
Leo	Jul 23–Aug 22
Virgo	Aug 23–Sep 22
Libra	Sep 23–Oct 22
Scorpio	Oct 23–Nov 21
Sagittarius	Nov 22–Dec 21

Display the sign, element, symbol, personality description, and simplified ASCII art corresponding to the selected sign.

```
=======================================
         YOUR ZODIAC SIGN
=======================================
              /\_/\
             ( o.o )
              > ^ <
Zodiac Sign: LEO
Element: Fire
Symbol: Lion
Personality: Confident, creative, courageous.
```

4. Horoscope Menu

```
[1] Love and Relationships
[2] Career and Studies
[3] Health and Wellness
[4] Lucky Number and Color
[5] View Zodiac Information
[6] Exit
Enter your choice:
```

Choices 1–3 display category-specific fictional predictions; choice 4 displays a lucky number and color; choice 5 shows zodiac details and art; choice 6 exits. Invalid selections must display an error and return to the menu.

5. Sample Interaction

```
Enter your name: Maria
Enter birth month: 8
Enter birth day: 15
Hello, Maria!
Your Zodiac Sign: LEO
Your Element: FIRE
Choice: 2
CAREER HOROSCOPE
Leo, your creativity will help you
solve a difficult problem today.
```

```
Stay focused and confident!
Return to menu? (Y/N): Y
```

Required Procedures

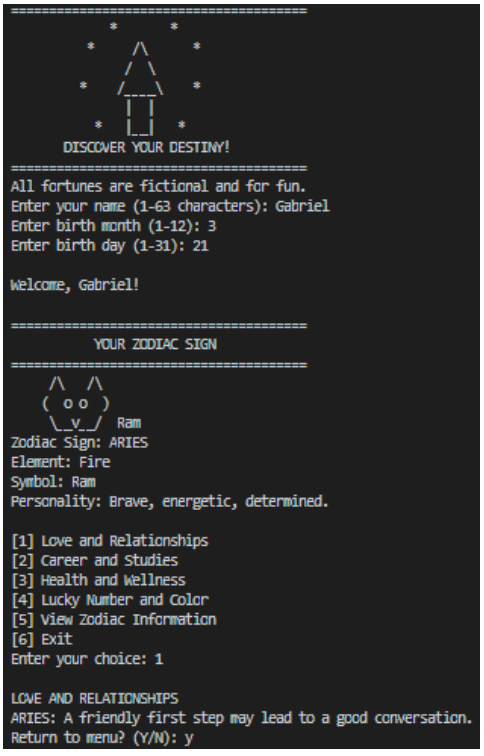
Procedure	Responsibility
display_welcome	Display welcome ASCII art
get_user_info	Read name and birth date
validate_birthdate	Check date validity
determine_zodiac	Identify zodiac sign
display_zodiac_art	Display sign-specific ASCII art
display_menu	Display horoscope choices
show_horoscope	Select and show prediction
exit_program	Terminate program

Required branching: CMP, JE/JNE for menus, and JL/JG/JLE/JGE for date ranges; include a menu loop.

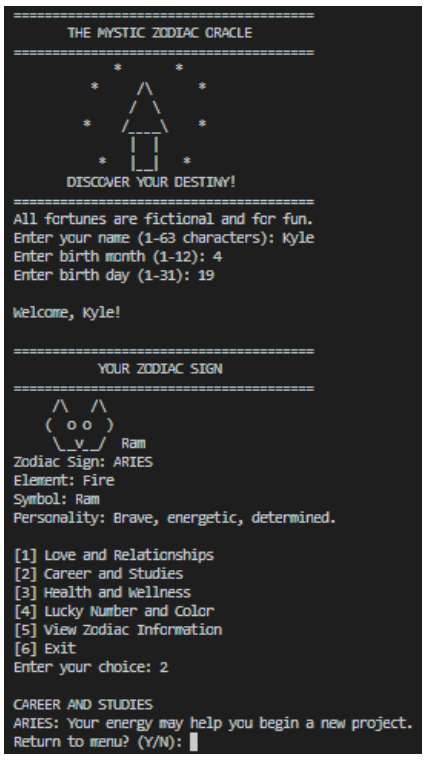
Test Cases

Test	Input	Expected result
1	March 21	Aries
2	April 19	Aries
3	April 20	Taurus
4	June 15	Gemini
5	July 23	Leo
6	August 15	Leo
7	December 25	Capricorn
8	February 30	Invalid date
9	Month 13	Invalid month
10	Menu choice 9	Invalid selection

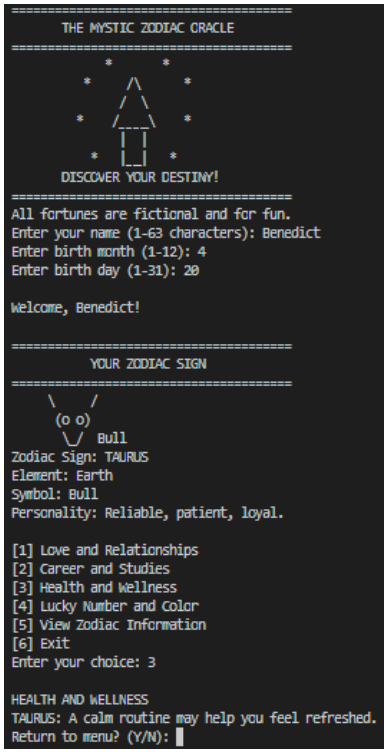
Test 1:



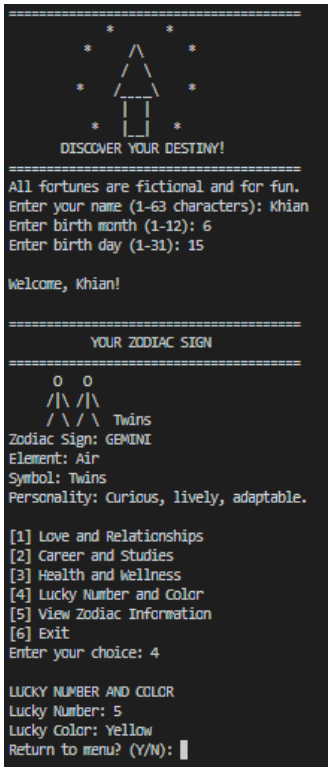
Test 2:



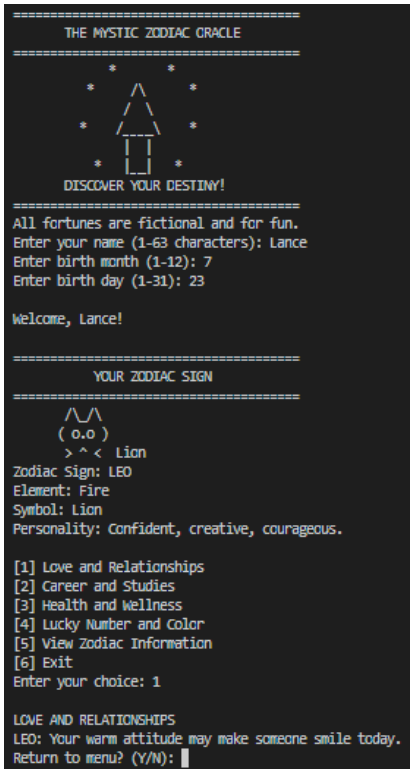
Test 3:



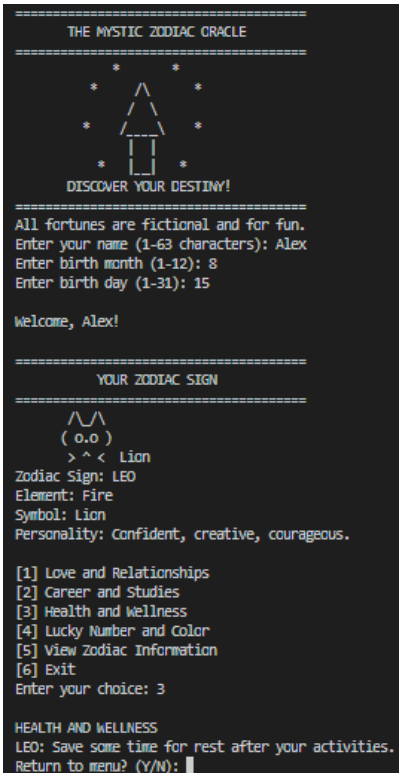
Test 4:



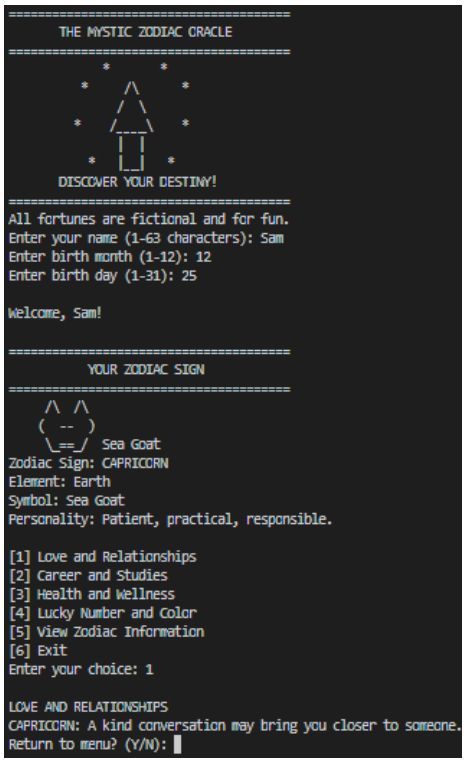
Test 5:



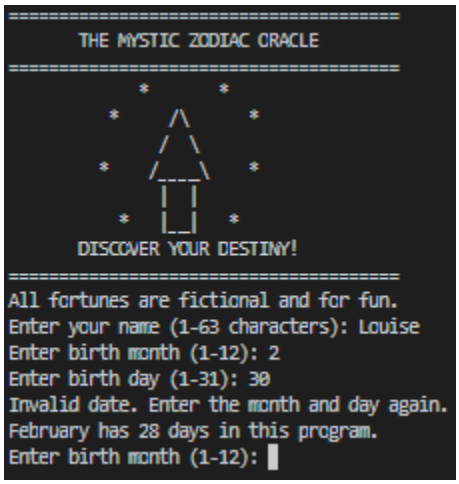
Test 6:



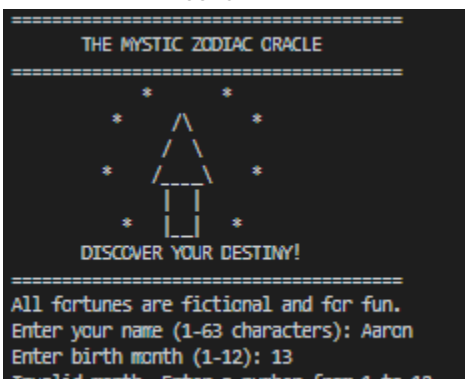
Test 7:



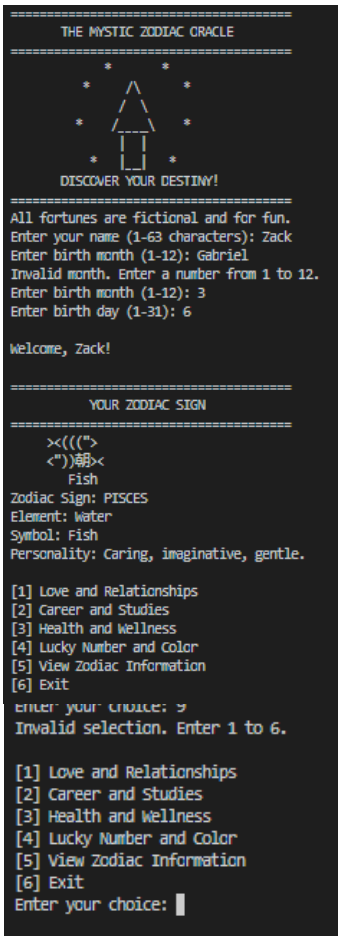
Test 8:



Test 9:



Test 10:



Machine Problem 2 — Dungeon Adventure and Survival Game

Problem Statement

Create a text-based Dungeon Adventure and Survival Game. The player selects a class, explores three rooms, battles enemies, collects treasure, and makes decisions affecting survival. Use procedures, conditional selections, arithmetic, loops, user input, and ASCII art.

1. Welcome Screen

```
========================================
       DUNGEON OF THE LOST KING
========================================
               |>>>>
               |
          _  _|_  _
         |;|_|;|_|;|
         \.    .  /
          \:  .  /
           ||:   |
           ||:.  |
           ||:  .|
       ENTER IF YOU DARE!
========================================
```

2. Player Registration

```
Enter your hero name: Arthur
Choose your character:
[1] Warrior
[2] Mage
[3] Archer
Enter choice: 1
Welcome, Warrior Arthur!
```

Character	Health	Attack	Potions
Warrior	100	20	2
Mage	70	30	3
Archer	85	25	2

3. Main Menu

```
[1] Explore Dungeon
[2] View Character Status
[3] Use Health Potion
[4] View Inventory
[5] Exit Game
```

4. Dungeon Rooms and Gameplay

Room 1 — Goblin Chamber: Goblin health = 30. Display enemy ASCII art and offer Attack, Defend, Use Potion, or Run.

```
          ,      ,
         /(.-""-.)\
     |\  \/      \/  /|
```

```
     | \ / =.  .= \ / |
      \  \   ^    /  /
       \  '-----'  /
A GOBLIN APPEARS!
[1] Attack  [2] Defend
[3] Use Potion  [4] Run
```

Room 2 — Treasure Chamber: The player chooses to open the chest (gain 50 gold) or leave it (gain nothing).

```
          __________
         /_________/|
        |  _____  | |
        | |  ?  | | |
        | |_____| | /
        |_________|/
[1] Open Chest  [2] Leave Chest
```

Room 3 — Dragon's Lair: Dragon health = 60. Show dragon ASCII art and the same combat menu as the goblin.

```
                / \  //\
      |\___/|      \
      /0  0  \       |
     /   ^    \     /
    /  \___/   \  /
    \___________\/
THE DRAGON AWAKENS!
```

5. Combat and Outcome Rules

Attack: subtract player attack points from enemy health. Defend: reduce the next enemy attack by 5 (minimum damage 0). Potion: restore 20 health up to starting maximum. Run: leave encounter and return to menu without advancing. A surviving goblin deals 10 damage per turn; a surviving dragon deals 15. After Attack or Defend, the living enemy retaliates. A player loses at health 0 or less; victory occurs after defeating the dragon.

```
VICTORY!
CONGRATULATIONS, ARTHUR!
You defeated the dragon and escaped!
Final Gold: 50
Remaining Health: 35
YOU ARE A HERO!

GAME OVER
Your hero has fallen.
The dungeon remains undefeated.
BETTER LUCK NEXT TIME!
```

Required Procedures

Procedure	Responsibility
display_welcome	Show title and ASCII art
get_player_info	Read name and class
initialize_character	Assign stats
display_main_menu	Show options
explore_dungeon	Control room progression
battle_enemy	Handle combat selections
use_potion	Restore health
display_status	Show statistics

display_inventory	Show gold and potions
display_game_result	Show victory or defeat
exit_program	Terminate program

Validate class selection, menus, potion count, health boundaries, room progression, and win/loss conditions. Never allow negative displayed health or potions below zero.

Test Cases

Test	Scenario	Expected result
1	Select Warrior	Health 100, Attack 20
2	Select Mage	Health 70, Attack 30
3	Select Archer	Health 85, Attack 25
4	Select class 9	Invalid selection
5	Warrior attacks Goblin twice	Goblin defeated
6	Potion at full health	Health capped; potion consumed
7	Potion count 0	Error; no health change
8	Open treasure	Gain 50 gold
9	Health reaches 0	Game Over
10	Dragon health reaches 0	Victory screen

TEST CASES MACHINE PROBLEM 2

TEST 1

```
=== CHARACTER STATUS ===
Hero Name: Zackv
Class: Warrior
Health: 100 / 100
Attack Power: 20
```

TEST 2

```
=== CHARACTER STATUS ===
Hero Name: Zackv
Class: Mage
Health: 70 / 70
Attack Power: 30
```

TEST 3

```
=== CHARACTER STATUS ===
Hero Name: Zackv
Class: Archer
Health: 85 / 85
Attack Power: 25
```

TEST 5

```
----------------------------------------

Enter your hero name: Zackv
Choose your character:
[1] Warrior
[2] Mage
[3] Archer
Enter choice: 9

Invalid selection! Please try again.
Choose your character:
[1] Warrior
[2] Mage
[3] Archer
Enter choice:
```

```
        ,     ,
       /(.-""-.)\
   |\  \/      \/  /|
   | \ / =.  .= \ / |
    \  \   ^    /  /
     \  '-----'   /
A GOBLIN APPEARS!

[1] Attack  [2] Defend
[3] Use Potion  [4] Run
Action: 1
You hit the enemy for 20 damage!
The enemy attacks you for 10 damage!

[1] Attack  [2] Defend
[3] Use Potion  [4] Run
Action: 1
You hit the enemy for 20 damage!
Goblin defeated!
```

TEST 6

```
You used a health potion and restored up to 20 HP!

 ====================================
                MAIN MENU
 ====================================
 [1] Explore Dungeon
 [2] View Character Status
 [3] Use health Potion
 [4] View Inventory
 [5] Exit Game
 Enter Choice: 3

Health already at maximum! Potion retained.
```

TEST 7

```
You used a health potion and restored up to 20 HP!
The enemy attacks you for 15 damage!

[1] Attack  [2] Defend
[3] Use Potion  [4] Run
Action: 3

No health potions remaining!
The enemy attacks you for 15 damage!

[1] Attack  [2] Defend
[3] Use Potion  [4] Run
Action:
```

TEST 8

```
        _________
       /________/|
      |  _____  | |
      | |  ?  | | |
      | |_____| | /
      |_________|/
YOU FOUND A TREASURE CHEST!

[1] Open Chest  [2] Leave Chest
Choice: 1

You opened the chest and found 50 Gold!
```

```
[1] Attack  [2] Defend
[3] Use Potion  [4] Run
Action: 1
You hit the enemy for 20 damage!
Dragon defeated!

 ====================================
               VICTORY!
 ====================================
CONGRATULATIONS, Zackv!
You defeated the dragon and escaped!
Final Gold: 50
Remaining Health: 75
YOU ARE A HERO!
 ====================================
```

TEST 9

```
The enemy attacks you for 15 damage!

 ====================================
               GAME OVER
 ====================================
Your hero has fallen.
The dungeon remains undefeated.
BETTER LUCK NEXT TIME!
 ====================================
```

Assessment Rubric

Criterion	Points
Correct procedures (CALL/RET)	25
Conditional selection and branching	20
User input and validation	15
Correct logic and output	15
ASCII art and interface creativity	10
Looping and menu navigation	10
Readability and comments	5
TOTAL	100

Required Student Deliverables

Submit the .asm source file, screenshots of successful execution and invalid-input handling, a short demonstration video, and a brief explanation of implemented procedures.

Open-Ended Questions

1. What is the purpose of procedures in your application?
2. How do CALL and RET control program execution?
3. Which conditional jump instructions did you use, and why?
4. How does your program handle invalid user input?
5. What would be harder to maintain without procedures?

Application-Specific Open-Ended Questions

Answer all questions for the application you developed. Write 3–5 complete sentences per question, refer to specific procedures or instructions from your own source code, and provide examples where requested.

Machine Problem 1 — Zodiac Horoscope and Fortune Teller

1. Explain how your determine_zodiac procedure identifies the correct sign from the birth month and day. Describe at least two conditional comparisons.

- The determine_zodiac procedure identifies the correct sign by checking the birth date to the when the zodiac sign changes.

2. How does your program handle zodiac date ranges that cross two months, such as Capricorn? Explain the branching logic.

- The program checks each month like for Capricorn. Days before January 20 is Capricorn and later days is Aquarius while December 22 and later is Capricorn.

3. Why did you divide input collection, date validation, zodiac identification, and horoscope display into separate procedures?

- It is separated because separate procedures make the code easier to do, fix, and understand.

4. Describe how your program rejects invalid dates such as February 30 and an invalid month such as 13.

- The program will show an error message and will loop again if an invalid month or date has been inputted.

5. How does the program select a horoscope category after the user enters a menu choice? Identify the conditional jump instructions used.

- The program checks the chosen number and it will show the matching horoscope for it.

6. How did you implement the zodiac ASCII art? Explain how the correct design is selected and displayed.

- The ASCII art is stored as a text, and it will show if it matches the choice.

7. What happens when a user enters an invalid menu option? Explain how control returns to the menu.

- When a user enters an invalid menu option, it will show an error message, and the menu will appear again.

8. If you added a new horoscope category, which procedures and conditional branches would you modify? Explain why.

- I would update the menu text and move the exit up since I would add another new horoscope category.

Machine Problem 2 — Dungeon Adventure and Survival Game

1. Explain how initialize_character assigns different health, attack, and potion values for Warrior, Mage, and Archer.

- initialize character checks class_id using cmp and jumps to the matching label to load the values of their health, attack, and potions. Warrior gets 100 hp/ 20 atk 2 potions, Mage gets 70 hp/30 At/ 3 potions, and archer gets 85 hp / 25 atk / Potions.

2. Describe the procedures and conditional jumps used when the player chooses Attack, Defend, Use Potion, or Run.

- Attack does damage to the enemy depending on its attack_power *from* enemy_hp and calls enemy_retaliate.

- defend is to mitigate the incoming damage, then calls enemy_retaliate.

- Potion calls usepotion before triggering enemy_retaliate.

- Run the player will display a flee message and return to the main menu loop.

3. How does your battle_enemy procedure determine whether the player or enemy has been defeated? Explain the comparisons.

-After

4. Explain how you prevent health from exceeding the maximum and prevent potion use when the inventory is empty.

- After every turn, it compares both health values to 0. If the enemy's health hits 0 or below, you win; if your health hits 0 or below, it triggers the game over screen.

5. How is the current dungeon room tracked, and what conditions allow the player to advance to the next room?

- A variable called current_room keeps track of your room number (1 for Goblin, 2 for Chest, 3 for Dragon). Winning a battle or finishing a room event increases this number, so you can move forward.

6. What is the purpose of separating display_status, display_inventory, and battle_enemy into different procedures?

- It keeps the code clean, organized, and easy to fix. This lets you check your stats or inventory anytime without accidentally triggering combat logic.

7. How does your program preserve important register values across procedures and Linux system calls? Give one example.

- Procedures use push to temporarily store important registers on the stack and pop to restore them before ending. For example, print_str saves eax, ebx, ecx, and edx so system calls don't overwrite your game's working data.

8. If you introduced a fourth room with a stronger enemy, what program data, procedures, and conditions would need to change?

- You would need to create ASCII art and text for the new monster, add a .room_4 section in explore_dungeon, and change Room 3 so defeating the Dragon moves current_room to 4 instead of ending the game.

Submission Requirement for Written Responses

Include a separate “Open-Ended Questions” section in your submission. Answer the five general questions and the eight questions for your assigned machine problem. Answers must reflect your actual program implementation; unsupported or generic explanations will not receive full credit.

Instructor Guidance

Assign Zodiac Horoscope first to focus on procedures, branching, and input validation. Assign Dungeon Adventure afterward for arithmetic, game state, repeated decisions, and interacting procedures.